Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-25

Бекназарова А. С.

Проверил:

Хацкевич А. С.

Брест 2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Вариант 3

Целевой системный вызов (или tracepoint): sys_enter_fchmodat**, Условие аномального поведения (Фильтр ядра):** Попытка завершить процесс с `PID = 1` (init/systemd).

Ход работы

Задание 1: базовый перехват запуска процессов.

Код:

```
from bcc import BPF

bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
};

BPF_PERF_OUTPUT(events);

int syscall__execve(struct pt_regs *ctx) {
    struct data_t data = {};
    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid();
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    events.perf_submit(ctx, &data, sizeof(data));
    return 0;
}
"""

b = BPF(text=bpf_text)
execve_fnname = b.get_syscall_fnname("execve")
b.attach_kprobe(event=execve_fnname, fn_name="syscall__execve")

print("[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)")

def print_event(cpu, data, size):
    event = b["events"].event(data)
```

```
    print(f"Перехват!    PID:    {event.pid},    UID:    {event.uid},    Процесс:
{event.comm.decode('utf-8', 'replace')}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

Результат:

```
[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)
Перехват! PID: 16581, UID: 0, Процесс: (-service)
Перехват! PID: 16588, UID: 0, Процесс: wsl-pro-service
Перехват! PID: 16595, UID: 0, Процесс: wsl-pro-service
Перехват! PID: 16596, UID: 0, Процесс: wsl-pro-service
Перехват! PID: 16597, UID: 0, Процесс: bash
Перехват! PID: 16596, UID: 0, Процесс: bash
Перехват! PID: 16598, UID: 0, Процесс: cloud-init
Перехват! PID: 16598, UID: 0, Процесс: cloud-init
Перехват! PID: 16598, UID: 0, Процесс: cloud-init
Перехват! PID: 16598, UID: 0, Процесс: cloud-init
Перехват! PID: 16599, UID: 0, Процесс: cloud-init
Перехват! PID: 16599, UID: 0, Процесс: cloud-init
Перехват! PID: 16599, UID: 0, Процесс: cloud-init
Перехват! PID: 16599, UID: 0, Процесс: cloud-init
Перехват! PID: 16600, UID: 0, Процесс: wsl-pro-service
Перехват! PID: 16614, UID: 1000, Процесс: Relay
Перехват! PID: 16615, UID: 1000, Процесс: bash
Перехват! PID: 16616, UID: 1000, Процесс: bash
Перехват! PID: 16619, UID: 1000, Процесс: bash
Перехват! PID: 16620, UID: 1000, Процесс: bash
Перехват! PID: 16621, UID: 1000, Процесс: bash
Перехват! PID: 16622, UID: 1000, Процесс: bash
Перехват! PID: 16623, UID: 1000, Процесс: lesspipe
Перехват! PID: 16625, UID: 1000, Процесс: lesspipe
Перехват! PID: 16626, UID: 1000, Процесс: bash
Перехват! PID: 16627, UID: 1000, Процесс: bash
Перехват! PID: 16628, UID: 1000, Процесс: bash
^C
[*] Завершение работы монитора.
```

Для генерации событий в соседнем терминале запускались стандартные утилиты. eBPF-зонд успешно перехватил вызовы и вывел данные процессов (PID, UID, имя процесса) в консоль.

Задание 2: динамический анализ аномального поведения.

Код:

```
from bcc import BPF
```

```
bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
    char path[256];
};

BPF_PERF_OUTPUT(events);

// Перехватываем sys_enter_fchmodat
TRACEPOINT_PROBE(syscalls, sys_enter_fchmodat) {
    struct data_t data = {};

    // ФИЛЬТРАЦИЯ В ЯДРЕ: проверяем, что запрашиваются права 0777
    if ((args->mode & 0777) == 0777) {
        data.pid = bpf_get_current_pid_tgid() >> 32;
        data.uid = bpf_get_current_uid_gid();
        bpf_get_current_comm(&data.comm, sizeof(data.comm));
        // Безопасно копируем путь к файлу
        bpf_probe_read_user_str(&data.path, sizeof(data.path), args->filename);

        // Отправляем в Python только если это аномалия
        events.perf_submit(args, &data, sizeof(data));
    }
    return 0;
}
"""

b = BPF(text=bpf_text)
print("[*] Мониторинг аномалий запущен...")

def print_event(cpu, data, size):
    event = b["events"].event(data)
    path_str = event.path.decode('utf-8', 'replace')
    comm_str = event.comm.decode('utf-8', 'replace')
    print(f"Предупреждение! Попытка выдать права 0777: PID: {event.pid}, UID:
{event.uid}, Процесс: {comm_str}, Путь: {path_str}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

Результат:

```
a@DESKTOP-SRBVTD9:~$ touch test_anomaly.txt
a@DESKTOP-SRBVTD9:~$ chmod 777 test_anomaly.txt

[*] Мониторинг аномалий запущен. Ожидание установки прав 0777... (Нажмите Ctrl+C для выхода)
Предупреждение! Попытка выдать права 0777: PID: 16757, UID: 1000, Процесс: chmod, Путь: test_anomaly.txt
```

В соседнем терминале была сгенерирована целевая аномалия: создан файл test_anomaly.txt, после чего к нему была применена команда chmod 777 test_anomaly.txt (попытка установить максимальные права доступа). eBPF-зонд успешно зафиксировал системный вызов sys_enter_fchmodat, применил фильтр на стороне ядра и вывел алерт с указанием PID, UID, имени процесса (chmod) и пути к файлу.

Задание 3: интеграция с пространством пользователя и аудит.

Код:

```
from datetime import datetime

def print_event(cpu, data, size):
    event = b["events"].event(data)
    path_str = event.path.decode('utf-8', 'replace')
    comm_str = event.comm.decode('utf-8', 'replace')

    # Формирование временной метки
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    alert_msg = f"Предупреждение! Попытка выдать права 0777: PID: {event.pid}, UID: {event.uid}, Процесс: {comm_str}, Путь: {path_str}"
    print(alert_msg)

    # Запись в лог-файл
    with open("security_audit.log", "a") as f:
        f.write(f"[{timestamp}] PID: {event.pid}, UID: {event.uid}, Процесс: {comm_str}, Путь: {path_str}\n")
```

Результат:

```
a@DESKTOP-SRBVTD9:~$ chmod 777 test_anomaly.txt
a@DESKTOP-SRBVTD9:~$ cat security_audit.log
[2026-11-10 14:49:50] PID: 16831, UID: 1000, Процесс: chmod, Путь: test_anomaly.txt

[*] Мониторинг аномалий с логированием запущен...
Предупреждение! Попытка выдать права 0777: PID: 16831, UID: 1000, Процесс: chmod, Путь: test_anomaly.txt
```

После запуска обновленного скрипта была сгенерирована аномалия (выдача прав 777). В файл security_audit.log успешно добавилась запись, содержащая временную метку (Timestamp), PID

нарушителя, UID, имя процесса и путь к файлу. Это подтверждает успешную интеграцию eBPF-зонда с пространством пользователя и систему аудита.

1) Сравнение Seccomp-BPF и eBPF (kprobes):

Механизм Seccomp-BPF предназначен для строгого ограничения (песочницы) и работает на этапе входа в системный вызов. Он имеет легальную возможность отменить вызов, вернув процессу код ошибки (например, EPERM), что делает его идеальным для блокировки нежелательных действий.

В отличие от него, eBPF (kprobes/tracepoints) — это механизм трассировки и мониторинга. Зонд eBPF прикрепляется к функции ядра и срабатывает уже внутри системного вызова, когда изменения состояния системы начались. У eBPF-зонда нет легального способа отменить вызов (вернуть ошибку). Если попытаться прервать системный вызов из eBPF через изменение регистров процессора, это нарушит внутренние инварианты ядра. Поскольку зонд вмешивается в уже выполняющийся вызов, это может привести к нестабильности ОС, утечкам памяти или панике ядра. Поэтому стандартные eBPF-зонды используются именно для аудита и обнаружения аномалий, а не для их блокировки.

2) Атака TOCTOU (Time-of-Check to Time-of-Use):

Это концепция состояния гонки, при которой состояние ресурса изменяется между моментом его проверки и моментом использования. В контексте eBPF-зонда из Задания 2 (перехват fchmodat), зонд считывает путь к файлу из памяти пользователя (по указателю args->filename) для проверки. Злоумышленник может использовать многопоточность: пока eBPF-программа проверяет безопасный путь, вредоносный поток подменяет строку в памяти по этому же указателю на другой (опасный) путь. В результате проверка проходит для одного файла, а ядро выполняет операцию над другим.

Для противодействия этой уязвимости при разработке eBPF-программ применяется безопасное копирование памяти. Вместо прямого чтения указателя пользователя, используется функция bpf_probe_read_user_str(). Она атомарно копирует данные из пространства пользователя в локальный буфер внутри eBPF-программы (в стек ядра). Дальнейшая проверка происходит уже с этой неизменяемой копией, которую злоумышленник не может подменить извне.

Вывод: в данной лабораторной работе я освоила методы разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC.